

Learning on Graphs

Homework 1

May 15, 2026

Tom Yachil 326192846

1 Part A: Theory warm up

A1. Permutation Equivariance

By definition:

$$H' = \tilde{A}'X'W = \tilde{A}'PXW$$

In addition, We can see that:

$$PH = P\tilde{A}XW$$

So, If we assume $\tilde{A}' = P\tilde{A}P^T$, and by using $P^TP = I$ we get:

$$H' = \tilde{A}'PXW = P\tilde{A}P^TPXW = P\tilde{A}XW = PH$$

As required.

A2. Laplacian and Dirichlet energy

$$x^TLx = x^T(D - A)x = x^TDx - x^TAx$$

By definition, D is a diagonal matrix whose entries satisfy $D_{ii} = \sum_j A_{ij}$. Therefore, $x^TDx = \sum_{i=1}^n \sum_{j=1}^n A_{ij}x_i^2$.

Now, by definition:

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

So plugging that definition to the first equation:

$$\begin{aligned} x^TLx &= \sum_{(i,j) \in E} x_i^2 + \sum_{(i,j) \in E} x_i x_j = \frac{1}{2} \left(2 \sum_{(i,j) \in E} x_i^2 + \sum_{(i,j) \in E} 2x_i x_j \right) = \\ &= \frac{1}{2} \left(\sum_{(i,j) \in E} x_i^2 + \sum_{(i,j) \in E} x_j^2 + \sum_{(i,j) \in E} 2x_i x_j \right) = \frac{1}{2} \sum_{(i,j) \in E} (x_i - x_j)^2 \end{aligned}$$

Dirichlet energy essentially measures the variation of the signal between connected nodes. Smaller the energy means smaller variation which is characterized by a smooth graph signal.

A3. Locality of polynomial filters

Given:

$$p_K(L)x = \sum_{k=0}^K \theta_k L^k x$$

Let us examine $(Lx)_i$:

$$(Lx)_i = (Dx - Ax)_i = \left(\sum_{(i,j) \in E} x_i - \sum_{(i,j) \in E} x_j \right)_i = \left(\sum_{(i,j) \in E} x_i - x_j \right)$$

So $(Lx)_i$ depends only on x_i and its 1-hop neighbors. By induction, $(L^k x)_i$ depends only on the k -hop neighborhood, so $(p_K(L)x)_i$ depends only on the K -hop neighborhood of i .

A4. Aggregation and Multiplicity

Given: $M_1 = \{\{a, b\}\}$ $M_2 = \{\{a, a, b, b\}\}$.

1. $\text{mean}(M_1) = \frac{a+b}{2} = \frac{2a+2b}{4} = \text{mean}(M_2)$
2. Given $a > b$, $\max(M_1) = a = \max(M_2)$
3. only sum can preserve multiplicity information well enough to match 1-WL, as can be seen: $\text{sum}(M_1) = a + b \neq 2a + 2b = \text{sum}(M_2)$

2 Part B: NetworkX graph operators

B1. Load and inspect Karate Club

Statistics computed from `nx.karate_club_graph()`:

nodes n	34
edges m	78
average degree	4.588
density	0.1390
connected components	1
min / max degree	1 / 17

The two club labels are **Mr. Hi** and **Officer**, with 17 nodes each. Figure 1 (left) shows the graph drawn with a spring layout, nodes colored by their club attribute.

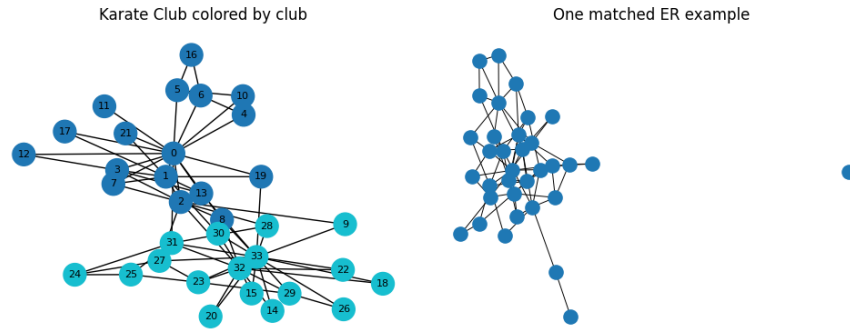


Figure 1: Karate Club graph colored by the `club` attribute (left) and one matched Erdős–Rényi control graph (right).

B2. Generate comparable Erdős–Rényi graphs Using $n = 34$, $m = 78$, the matched edge probability is

$$p_{\text{match}} = \frac{2m}{n(n-1)} = 0.1390.$$

Three ER graphs were generated with `nx.erdos_renyi_graph(34, 0.1390, seed)` for seeds 1, 2, 3. The combined statistics are:

graph	n	m	avg deg	density	min deg	max deg	#comp
Karate	34	78	4.588	0.1390	1	17	1
ER_1	34	75	4.412	0.1337	0	9	2
ER_2	34	79	4.647	0.1408	0	9	2
ER_3	34	71	4.176	0.1266	1	10	1

See Figure 1 (right) for one ER visualization (note the isolated node) and Figure 2 for the degree histograms.

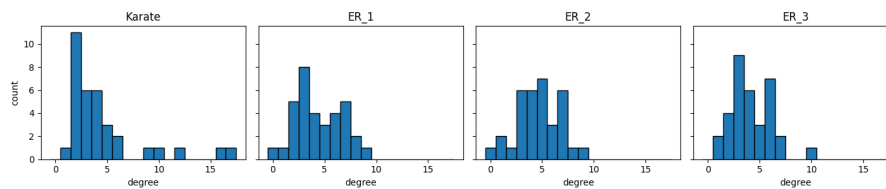


Figure 2: Degree histograms for Karate Club and the three matched Erdős–Rényi controls. All four panels use the same axis range to make the shapes directly comparable.

- 1. Are the ER graphs always connected?** No, connectivity is probabilistic. In our run, ER_1 and ER_2 have 2 components while ER_3 is connected, even though all three use the same (n, p) .
- 2. How different are the degree distributions?** Same mean (~ 4.59) by construction, but different shape:

- **Karate**: heavy right tail with hubs at degree 17 (Mr. Hi) and 16 (Officer), $\min = 1$.
- **ER**: roughly binomial / symmetric around the mean, max degree only ≈ 9 –10, no hubs.

Matching (n, p) matches the first moment but not the structure — real social networks have organized hubs that pure randomness does not reproduce.

3. Fixed Karate vs. random ER

- Karate is identical on every run, so results are reproducible, and it carries real labeled community structure.
- ER is a distribution; varying the seed (or n, p) gives controlled variation: multiple seeds quantify random fluctuation. Any gap between Karate and the density-matched ER samples is then attributable to structure beyond density.

B3. Matrix representations

All matrices were built directly from `nx.to_numpy_array(G_karate)` and routines we implemented by hand. Reported nnz counts:

matrix	shape	nnz	density
A	34×34	156	0.1349
D	34×34	34	0.0294
$L = D - A$	34×34	190	0.1644
$\tilde{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$	34×34	190	0.1644

Why dense adjacency matrices are not suitable for large sparse graphs?

In sparse graphs the density is much lower than 1. Therefore using dense storage is wasteful in memory (almost every entry is zero), and wasteful in compute. Since $D, L, \tilde{A}$ depend on the adjacency matrix, this dense overhead builds up.

B4. Graph signal smoothing

Define the bipolar signal

$$x_i^{(0)} = 1 - 2\text{karate.y}_i \in \{+1, -1\},$$

so Mr. Hi's club maps to +1 and the Officer's club to -1.

We iterate $x^{(k+1)} = \tilde{A}x^{(k)}$ for $k = 0, 1, \dots, 20$. Figure 3 shows the signal on the graph at $k \in \{0, 1, 5, 20\}$ with a shared color range, and Figure 4 shows the Dirichlet energy $x^{(k)\top} L x^{(k)}$ and the variance $\text{Var}(x^{(k)})$.

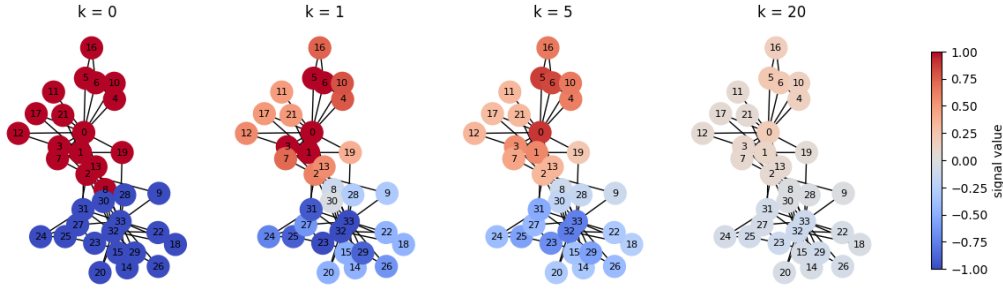


Figure 3: Signal $x^{(k)} = \tilde{A}^k x^{(0)}$ on the Karate graph at depths $k = 0, 1, 5, 20$, with shared color scale $[-1, +1]$.

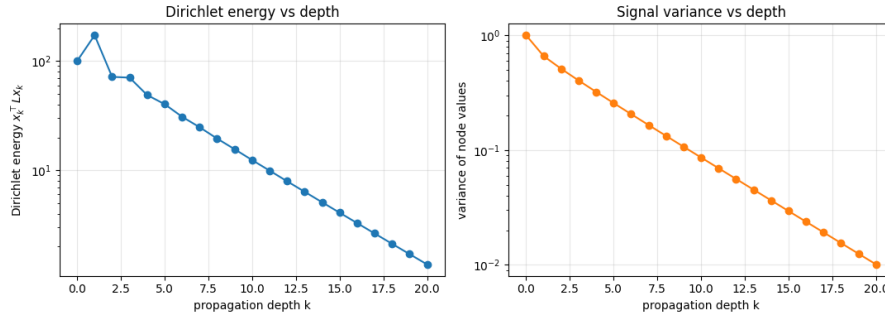


Figure 4: Dirichlet energy (left) and node-value variance (right) of $x^{(k)}$ as a function of propagation depth k , on log scale.

What happens as k increases? Both Dirichlet energy and variance decay monotonically: every eigenvalue of $\tilde{A}$ except $\lambda_1 = 1$ has $|\lambda_i| < 1$, so non-smooth components shrink geometrically and only the smoothest mode survives, i.e. the **oversmoothing** phenomenon.

3 Part C: Semi-supervised node classification with GCN on Cora

C1. Dataset inspection

Statistics computed from the Cora Planetoid dataset:

nodes	2,708
edges (undirected)	5,278
node features	1,433
classes	7
average degree	3.898
train / val / test nodes	140 / 500 / 1,000
train labels per class	20

Why is this a transductive node-classification setup? A single fixed graph with all 2,708 nodes, all edges, and all features is shared across train/val/test;

only the label masks differ. Message passing already sees unlabeled nodes during training, and predictions are made for every node of the same graph: the defining property of transductive node classification.

C2. MLP baseline

- **Architecture:**

$$X \rightarrow \text{Linear}(1433 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(64 \rightarrow 7)$$

- **Training:** Adam with $\text{lr} = 0.01$, $\text{weight_decay} = 5 \times 10^{-4}$, cross-entropy loss on the train mask, for exactly 500 epochs. The checkpoint with the best validation accuracy is kept and used to report test accuracy.
- **Results:** 92,231 trainable parameters. Best validation accuracy 0.5860 at epoch 82; test accuracy at that checkpoint 0.5710. Training curves are shown in Figure 5 (MLP traces).

C3. Two-layer GCN

- **Architecture:**

$$X \rightarrow \text{GCNConv}(1433 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{GCNConv}(64 \rightarrow 7)$$

Using `torch_geometric.nn.GCNConv`.

- **Training:** Same optimizer settings and 500-epoch schedule as the MLP.
- **Results.** 92,231 trainable parameters. Best validation accuracy 0.7900 at epoch 24; test accuracy at that checkpoint 0.7960. Training curves are shown in Figure 5 (GCN traces).

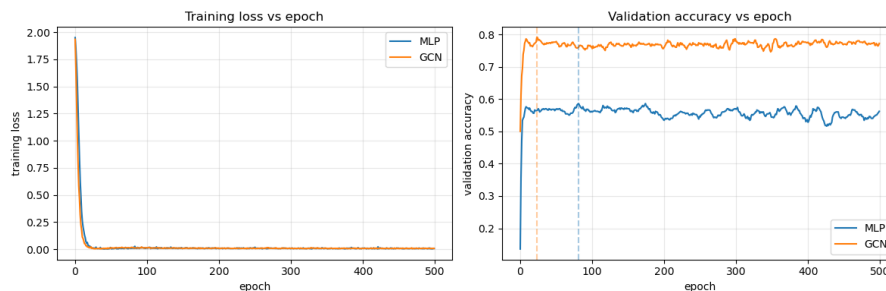


Figure 5: Training loss (left) and validation accuracy (right) over 500 epochs for the MLP and GCN on Cora. Dashed vertical lines mark the best-val checkpoint of each model (MLP: epoch 82, GCN: epoch 24).

C4. Results table and discussion

Model	Parameters	Best val. acc.	Test acc.	Notes
MLP	92,231	0.5860 (ep. 82)	0.5710	Ignores graph edges
GCN	92,231	0.7900 (ep. 24)	0.7960	Two <code>GCNConv</code> layers

The MLP treats each paper as an independent bag-of-words vector and ignores the citation graph, while the two **GCNConv** layers mix each node's features with its 2-hop neighborhood. This exploits Cora's homophily (cited papers tend to share a label) and lifts test accuracy from **57.1%** to **79.6%**, a 22-point gap that comes entirely from the graph. Both models have the same parameter count (92,231), so structural information is the dominant factor here, not model capacity. Both models also overfit: training accuracy hits 1.0 within ~ 50 epochs, the MLP's best-val checkpoint is at epoch 82 and the GCN's at epoch 24, and validation accuracy stagnates beyond those points. For the GCN the best-val (0.790) and test (0.796) accuracies agree within noise, so the selected checkpoint generalizes well.

4 Part D: Depth and oversmoothing with fixed GCN propagation

D1. Fixed GCN propagation on Cora

Starting from $H^{(0)} = X$ (Cora node features), we iterate $H^{(k+1)} = SH^{(k)}$ with $S = \widehat{D}^{-1/2} \widehat{A} \widehat{D}^{-1/2}$, $\widehat{A} = A + I$. Reported at the requested depths:

k	Dirichlet energy $\text{Tr}(H^{(k)\top} L H^{(k)})$	mean pairwise distance
0	1.6096×10^5	5.8109
1	3.5364×10^4	3.1777
2	1.4514×10^4	2.5217
4	1.1778×10^4	2.0414
8	1.0237×10^4	1.6033
16	9.1144×10^3	1.3000
32	8.3409×10^3	1.0352
64	7.8904×10^3	0.8073

D2. Cora smoothing diagnostic plots

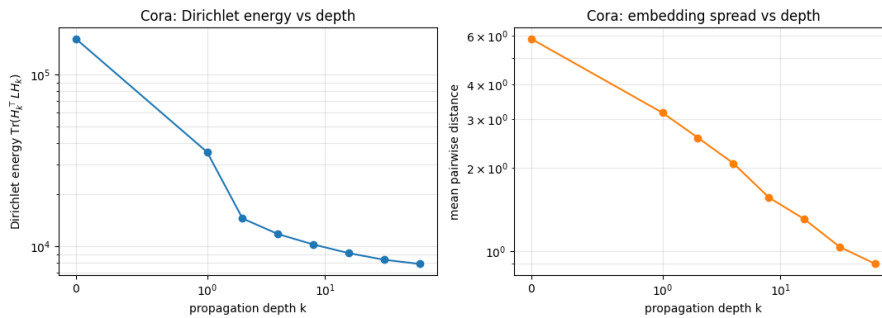


Figure 6: Dirichlet energy (left) and mean pairwise node-feature distance (right) on Cora as a function of propagation depth k , on log-log axes.

Both Dirichlet energy and mean pairwise distance decay roughly geometrically with k , and by $k = 64$ the embeddings are nearly indistinguishable - the oversmoothing effect.

D3. Provided ER oversmoothing visualization graph

I build S for the provided connected ER graph G_{over} ($n = 120$, $p = 0.06$, seed 7) and propagate the provided initial signal `over_signal0` (normalized to $[-1, 1]$) for the same depths.

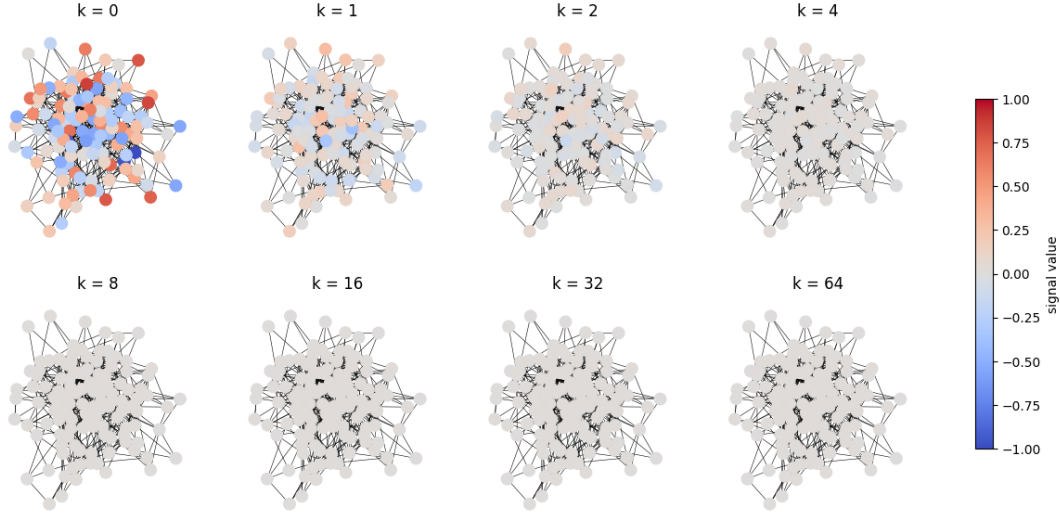


Figure 7: Signal $h^{(k)} = S^k h^{(0)}$ on G_{over} at depths $k = 0, 1, 2, 4, 8, 16, 32, 64$ with shared color range $vmin=-1$, $vmax=1$.

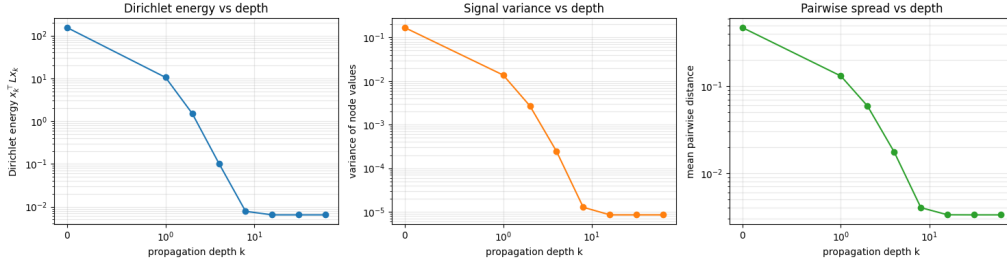


Figure 8: Dirichlet energy, signal variance, and mean pairwise distance on G_{over} as a function of propagation depth k , on log-log axes.

D4. Short reflection: range vs. discriminability

Each application of S mixes a node with its 1-hop neighbors, so after k steps every node has aggregated information from its k -hop neighborhood: the communication range grows linearly with depth. However, S has eigenvalues in $(-1, 1]$, so writing $H^{(k)} = \sum_i \lambda_i^k c_i u_i$, every non-leading coefficient decays as $|\lambda_i|^k \rightarrow 0$ and only the smooth u_1 mode survives. This matches the empirical curves on Cora and G_{over} : Dirichlet energy and mean pairwise distance both

decay roughly geometrically, and by $k = 64$ the embeddings are nearly indistinguishable.

5 Part E: 1-WL and expressivity toy study

The four toy graphs used in this part are shown in Figure 9.

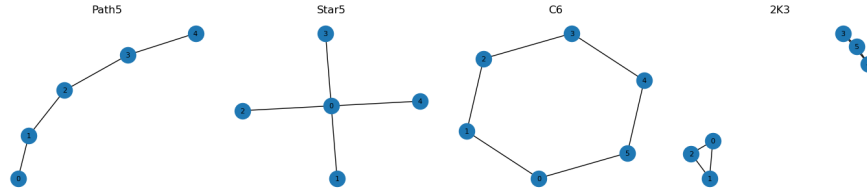


Figure 9: Toy graphs for the 1-WL study. Path5 and Star5 are a same-size warmup pair (5 nodes each); C_6 and $2K_3$ are the same-size failure pair (6 nodes each).

E1. Implement 1-WL color refinement

We implemented 1-WL color refinement: at each round, every node's new color is a deterministic function of its current color and the sorted multiset of its neighbors' current colors. Figure 10 shows the color-histogram evolution over rounds 0–3 for all four graphs. 1-WL distinguishes Path5 from Star5 (their multisets differ from round 1), but does *not* distinguish C_6 from $2K_3$ (both stay at a single color of multiplicity 6 forever).

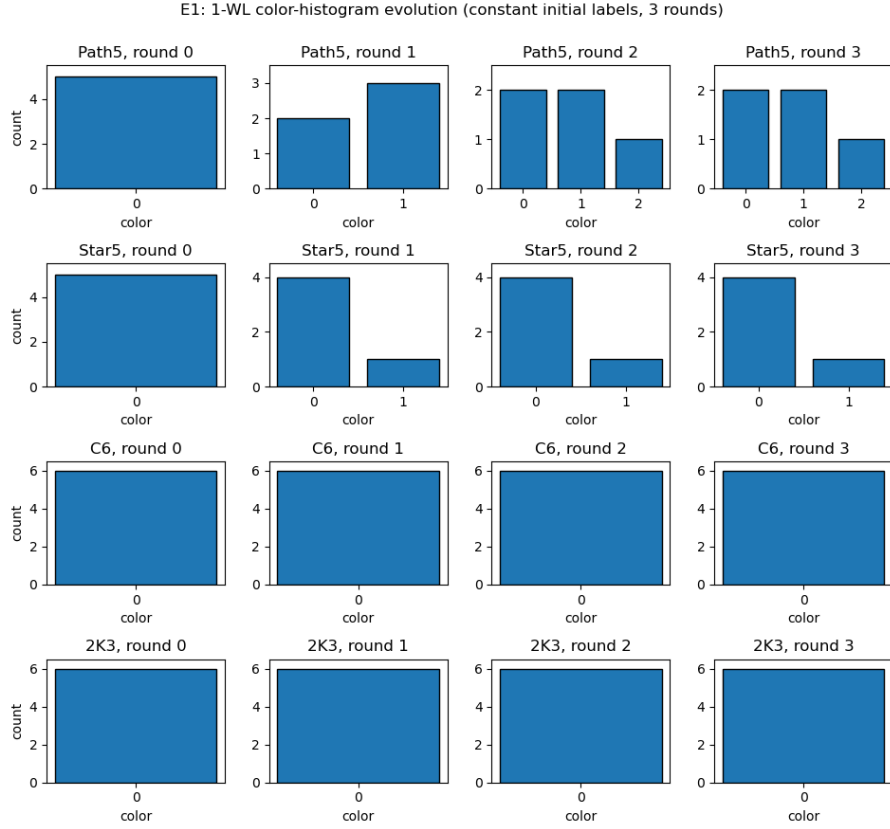


Figure 10: 1-WL color-histogram evolution under constant initial labels, rounds 0–3, for Path5, Star5, C_6 , $2K_3$.

E2. Failure case: C_6 versus $2K_3$

As already visible from Figure 10, C_6 and $2K_3$ both keep the histogram $\{0:6\}$ at every round, so 1-WL with constant initial labels cannot distinguish them.

As seen in lecture 2, both C_6 and $2K_3$ are 2-regular, so under constant initial labels every node has the same color as its two neighbors from round 1 onward. Every node therefore observes the same neighbor-color multiset and is updated to the same new color, leaving the histogram a single color of multiplicity 6 at every round. The same argument applies to any two r -regular graphs of the same size, so the failure is structural and independent of n .

E3. Structural augmentation with RWSE

We use Random-Walk Structural Encoding $\text{RWSE}_k(v) = (D^{-1}A)_{vv}^k$ for $k = 1, \dots, 6$ as initial node features. The return probabilities per node are constant within each graph (both are vertex-transitive) but differ between the graphs:

graph	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$
C_6	0	$1/2$	0	$3/8$	0	0.3438
$2K_3$	0	$1/2$	$1/4$	$3/8$	0.3125	0.3438

1. Does RWSE distinguish C_6 from $2K_3$? Yes. C_6 is bipartite, so $(P^k)_{vv} = 0$ for all odd k , while in $2K_3$ the 3-cycle gives $(P^3)_{vv} = 1/4$ at every node. The column $k = 3$ alone separates the two graphs because their RWSE feature multisets are different.

2. Why is RWSE more principled than arbitrary node IDs? RWSE is a function of the graph itself: $\text{RWSE}_k(v) = (D^{-1}A)^k_{vv}$ is determined by structure and is the same for isomorphic graphs. Node IDs are arbitrary labels, they distinguish anything trivially because they encode the labeling, not the structure, so a model using them is fitting the indexing rather than the graph.

3. Permutation equivariance / invariance? RWSE is permutation-equivariant at the node level: relabeling nodes by a permutation π permutes the RWSE feature vector by the same π , since $(P^\pi)_{\pi(v)\pi(v)}^k = (P^k)_{vv}$. As a consequence, the of RWSE features is permutation-invariant.

E4. Node coloring or numbering as a contrast case

We initialize C_6 and $2K_3$ with unique per-node IDs as colors (Figure 11).

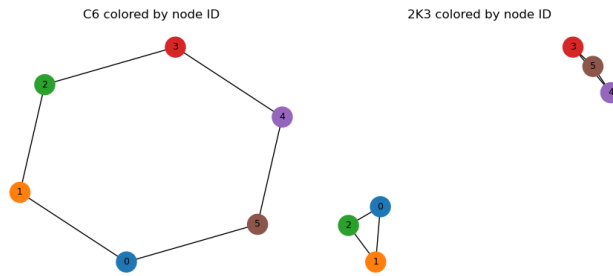


Figure 11: C_6 and $2K_3$ with each node colored by its unique ID.

What happens to the 1-WL histograms with unique IDs. Both C_6 and $2K_3$ have the histogram $\{1, 1, 1, 1, 1, 1\}$ (six singletons) at every round, and the signature (id, sorted neighbor ids) of every node is unique within and across the two graphs.

Why this trivially distinguishes the graphs. With n distinct labels, the initial coloring is already a perfect classifier of node identity, and refinement just confirms that. No information about the graph structure is needed: even comparing graphs that are isomorphic, using different ID assignments produces different histograms. The distinguishing power comes from labels, not topology.

Why this is not permutation-invariant or structurally meaningful. Node ID features are not permutation-invariant: the feature is the labeling, so under any relabeling the per-node feature changes to whatever π assigns, and the multiset of (id, neighborhood-of-ids) signatures changes too. The same C_6 graph with a random permutation of node IDs receives a completely different per-node coloring, even though the graph is unchanged.

6 Bonus: Oversquashing intuition on a barbell graph

We use the provided graphs G_{barbell} and $G_{\text{shortcut}} = G_{\text{barbell}} \cup \{(0, 19)\}$, shown in Figure 12.

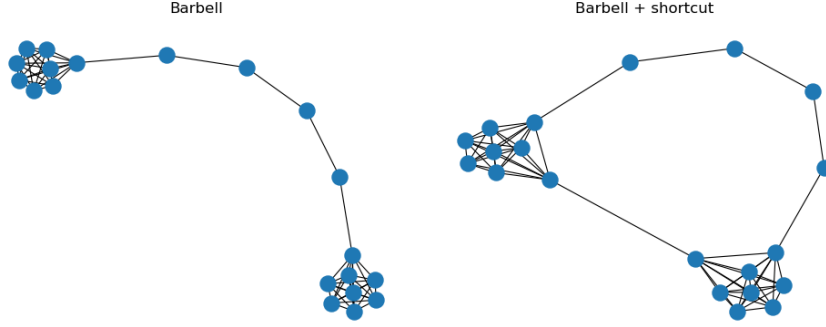


Figure 12: Barbell graph (left) and barbell with shortcut edge (0, 19) added (right).

F1. Bottleneck statistics

graph	diameter	avg. shortest path	λ_2 of L_{sym}	max edge betw. (edge)
barbell	7	3.758	0.00665	0.526 (9, 10)
barbell + shortcut	4	2.326	0.03481	0.421 (0, 19)

The shortcut almost halves the diameter and the average shortest path, raises the spectral gap λ_2 by a factor of ~ 5 , and the new edge (0, 19) becomes the highest-betweenness edge (replacing the original bridge (9, 10)).

F2. Signal transfer experiment

We inject $h^{(0)} = e_0$ (Dirac signal at node 0) and propagate $h^{(k+1)} = Sh^{(k)}$ with $S = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ for $K = 40$ steps. Figure 13 shows the mean $|h^{(k)}|$ on the right-clique nodes (12:20) and on the bridge/path nodes (8:12) for both graphs.

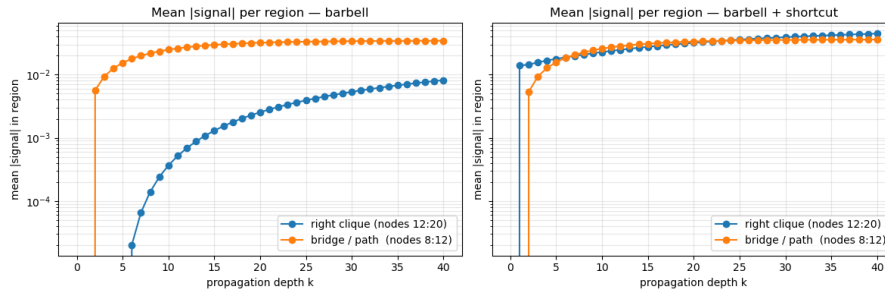
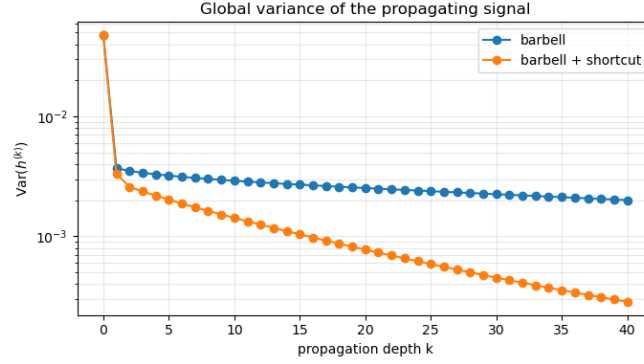
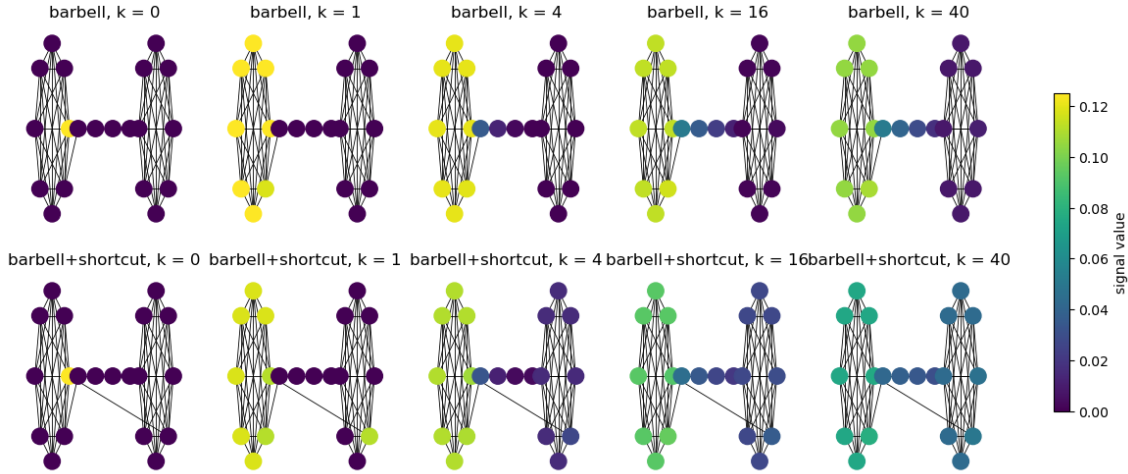


Figure 13: Mean $|h^{(k)}|$ on the right clique (nodes 12:20) and on the bridge/path (nodes 8:12) versus propagation depth k , for the plain barbell (left) and for the barbell with shortcut (right). Log scale on y .

F3. Smoothing and snapshots

Figure 14: Global variance $\text{Var}(h^{(k)})$ versus depth for both graphs (log scale on y).Figure 15: Signal snapshots at steps $k = 0, 1, 4, 16, 40$ for the plain barbell (top row) and the barbell with shortcut (bottom row), all panels sharing the same color range.

In the barbell, every message from the left clique to the right must squeeze through a single bridge edge, so the right-clique mean- $|signal|$ curve in Bonus 2 climbs very slowly (this is **oversquashing**: information through a bottleneck is exponentially damped). The shortcut $(0, 19)$ adds a second length-1 path between the cliques, so node 19 receives non-trivial mass already at $k = 1$ and the right-clique curve climbs orders of magnitude faster. The same edge also lowers the second-largest eigenvalue of S from ≈ 0.994 to ≈ 0.970 , so the variance (which decays geometrically at twice that rate per step) plummets and the $k = 16, 40$ snapshots look uniform while the plain barbell still shows a left-vs-right gradient. In short, the shortcut **alleviates oversquashing** but **accelerates oversmoothing**: both effects come from the same source.